



南开大学
Nankai University

南 开 大 学

计 算 机 学 院

并行程序设计实验报告

SIMD 相关实验及性能测试

林晖鹏

年级：2023 级

专业：计算机科学与技术

指导教师：王刚

2025 年 4 月 29 日

摘要

本文基于 ARM 架构阿里云 ECS 实例 (Yitian 710 芯片), 通过 Neon SIMD 指令集实现了 MD5 哈希算法的四路并行优化。实验表明, 在 O2 优化级别下, 四路并行版本相比串行版本获得 1.95 倍加速。通过 perf 性能分析工具揭示了分支预测失效 (205 万次 vs 190 万次) 和指令效率下降 (CPI 1.215 vs 0.694) 是主要性能瓶颈, 并尝试二路并行和八路并行进行对比, 探究多路并行效率。进一步在 x86 架构下采用 SSE 指令实现并行方案, O2 优化下获得 2.16 倍加速。跨架构对比显示, x86 平台在同等优化级别下展现出更优的 SIMD 加速潜力。代码仓库地址<https://gitee.com/absurdaaa/password-guessing.git>

关键字: SIMD 并行计算, MD5 优化, Neon 指令集, SSE 加速, 跨架构性能分析

目录

一、 基础要求	1
(一) Neon	1
(二) 算法设计及代码实现	1
1. 字符串获取	1
2. 转换为 Neon 向量	1
3. 逐个更新 block	2
(三) 性能测试	3
二、 进阶要求	5
(一) 不同优化对比	5
(二) 多路并行对比	6
(三) x86 框架下并行实现	6
1. SSE	6
2. 代码实现及结果	7
3. 结果分析	7
三、 总结	7

一、 基础要求

(一) Neon

Neon 是 ARM 架构中的 SIMD（单指令多数据）指令集扩展，专为高性能数据处理设计。它适用于多媒体处理、信号处理和机器学习等领域。

- **矢量操作：**支持 64 位和 128 位矢量寄存器。
- **数据类型：**支持整数和浮点类型，如 `int8x8_t`, `float32x4_t`。
- **并行计算：**在单个指令中处理多个数据元素。

(二) 算法设计及代码实现

在 MD5 哈希算法中，每个口令之间的处理是互不干扰的，我们利用 Neon 算法，将四个原始口令加载到一个 Neon 向量中，同时地并行处理多个口令，使得程序时间缩短，达到加速效果。比如一个 Neon 矢量为 128 位的矢量，如果由四个 32 位组成，在运算过程中，四个 32 位矢量互不干扰，这个特点为我们四个口令的并行计算提供了可行性。

1. 字符串获取

我们从猜测口令队列中一次性获取四个口令，如果不足四个口令了，就拿第一个口令填充。

```
1 // 确保内存 对齐
2 alignas(16) bit32 state[4][4];
3 for(int i = 0; i < q.guesses.size(); i+=4)
4 {
5     string pw[4];
6     pw[0] = q.guesses[i];
7     if(i+1<q.guesses.size())pw[1] = q.guesses[i+1];
8     else
9         pw[1] = q.guesses[i];
10    if(i+2<q.guesses.size())pw[2] = q.guesses[i+2];
11    else
12        pw[2] = q.guesses[i];
13    if(i+3<q.guesses.size())pw[3] = q.guesses[i+3];
14    else
15        pw[3] = q.guesses[i];
16    SIMD_MD5Hash(pw, state);
17 }
```

2. 转换为 Neon 矢量

我们先分别用 `StringProcess` 函数去处理读入的四个口令串，并记录每个口令的 block 数量以及最长口令串的 block 数量（用来结束某个口令的哈希计算），然后用 Neon 矢量去记录初始值，这里每个口令的初始值都是一样的。

```
1 Byte *paddedMessage[4];
2 int *messageLength = new int[4];
```

```

3   for (int i = 0; i < 4; i += 1) paddedMessage[i] = StringProcess(input[i],
    &messageLength[i]);
4   int n_blocks[4] = {messageLength[0] / 64, messageLength[1] / 64,
    messageLength[2] / 64, messageLength[3] / 64};
5   int max_n_blocks = n_blocks[0];
6   for (int i = 1; i < 4; i++){
7       if (n_blocks[i] > max_n_blocks) max_n_blocks = n_blocks[i];
8   }
9   for (int i = 0; i < 4; i++){
10      state[i][0] = 0x67452301;
11      state[i][1] = 0xefcdab89;
12      state[i][2] = 0x98badcfe;
13      state[i][3] = 0x10325476;
14  }
15  // 将变量类型修改为 Neon 向量类型
16  uint32x4_t a_end = vdupq_n_u32(state[0][0]);
17  uint32x4_t b_end = vdupq_n_u32(state[0][1]);
18  uint32x4_t c_end = vdupq_n_u32(state[0][2]);
19  uint32x4_t d_end = vdupq_n_u32(state[0][3]);
20  uint32x4_t a = a_end;
21  uint32x4_t b = b_end;
22  uint32x4_t c = c_end;
23  uint32x4_t d = d_end;

```

3. 逐个更新 block

1. 判断并保存口令哈希结果

在逐个计算 block 并更新最终结果的时候, 由于每个口令的长度可能不一致, 导致需要更新的 block 个数不一致, 但是这个 Neon 向量运算会每个 32 位都计算, 所以我们这里在每次计算 block 之前判断每个口令是否已经计算完所有 block (但是这个判断我们从), 如果已经计算完, 我们就保存 Neon 向量对应位置的结果. 下面给出一个例子:

```

1   if (i == n_blocks[0]) { // i 是循环轮次
2       state[0][0] = vgetq_lane_u32(a_end, 0);
3       state[0][1] = vgetq_lane_u32(b_end, 0);
4       state[0][2] = vgetq_lane_u32(c_end, 0);
5       state[0][3] = vgetq_lane_u32(d_end, 0);
6       over_num++; // 如果 over_num == 4, 结束循环
7   }

```

2. 计算 x[i]

在后续计算中用到的参数 x[i], 我们同样用 Neon 向量保存, 计算每个口令的 x[i], 写入对应位置. 这里考虑 offset 复用率, 所以用一个变量储存, 防止重复计算.

```

1   for (int i1 = 0; i1 < 16; ++i1){
2       int offset = 4 * i1 + i * 64;
3       x[i1] = (uint32x4_t){
4           *reinterpret_cast<uint32_t *>(paddedMessage[0] + offset),

```

```

5      *reinterpret_cast<uint32_t *>(paddedMessage[1] + offset),
6      *reinterpret_cast<uint32_t *>(paddedMessage[2] + offset),
7      *reinterpret_cast<uint32_t *>(paddedMessage[3] + offset) });
8  }

```

3. Neon 算法并行优化宏定义

剩下的代码部分和原本的思路没有太大差别, 接下来我们来修改 FF 这些宏定义, 改成 Neon 算法并行计算。我们可以观察到, 在原本的宏定义中, 有很多与、移位、加法操作, 这些操作就非常适合用来矢量的并行操作。

```

1      #define F(x, y, z) (((x) & (y)) | ((~x) & (z)))
2      ...
3      #define ROTATELEFT(num, n) (((num) << (n)) | ((num) >> (32-(n))))
4
5      #define FF(a, b, c, d, x, s, ac) { \
6      (a) += F((b), (c), (d)) + (x) + ac; \
7      (a) = ROTATELEFT((a), (s)); \
8      (a) += (b); \
9      }
10     ...

```

下面给出修改后的例子:

```

1      #define F(x, y, z) vorrq_u32(vandq_u32(x, y), vandq_u32(vmvnq_u32(x),
2      z))
3      ...
4      #define ROTATELEFT(num, n) vorrq_u32(vshlq_n_u32(num, n), vshrq_n_u32
5      (num, 32 - n))
6
7      #define FF(a, b, c, d, x, s, ac){ \
8      a = vaddq_u32(a, vaddq_u32( F(b, c, d) , vaddq_u32(x, vdupq_n_u32(
9      ac)))); \
10     a = vaddq_u32( ROTATELEFT(a, s) , b); \
11     }
12     ...

```

(三) 性能测试

至此, 我们实现了基本的 SIMD 算法, 一次性并行处理四个口令。接下来我们来性能测试。

我们执行对两个版本执行 main 函数 (不开优化), 得到的 hash 部分是时间分别为 **6.99188 s** 和 **6.50169 s**, 此时的加速比约为 **1.08** (这个加速比还收到系统波动的影响, 但是总体在 1.1 上下浮动)。实际上结果并没有达到四路并行的理想结果, 接下来我们来设计测试分析原因:

我们设计了四个口令, 分别用串行版本和并行版本对这四个口令执行 100000 次 (这里对比不进行 O1O2 优化), 用 perf 检测计算 CPI 指数、缓存命中率、分支预测等指标。结果如表1。

1. CPI 上升

虽然指令数减少了接近一半 (这里不是四倍关系是因为加载数据以及组合数据的花销增大), 但是 CPI 上升至 1.2, 说明每个指令花费的周期数增加, 因为 Neon 指令变复杂了。

2. 分支未命中率增加

在四路并行版本里面，由于每个指令可能长度不一致，导致计算 block 数目不同，每次计算的时候需要检查每个指令是否已经计算完成，需要保存数据。这些大量判断导致了分支未命中率增加，较高的分支未命中率会导致更多的重调度和恢复开销，降低 CPU 执行效率。

3. 缓存命中

对比总缓存和 L1 缓存，我们发现四路并行的总缓存命中率为 99.6%，而串行版本的总缓存命中率为 99.7%，其实是差别不大的。但是结合 L1 缓存命中数据，我们发现四路并行版本的 L2 缓存命中率稍微高一点，可能是同时处理四路的数据，导致很多缓存数据存在 L2 中能够更多的被复用。

从数据来看，并行版本在指令数和周期数上都有减少（理论上意味着并行化带来了计算合并和减少重复操作），但由于额外的数据组织、地址计算及增多的控制分支，导致每条指令的执行效率明显下降。

表 1: 串行版本 vs 并行版本

指标	串行	四路并行
Cycles	5,367,269,828	4,846,472,769
Instructions	7,728,455,975	3,988,338,467
CPI	0.694	1.215
Cache-References	3,720,125,045	5,471,341,424
Cache-Misses	1,209,924	1,968,763
Branches	246,592,189	105,629,888
Branch-Misses	1,900,674	2,056,406
L1-dcache-loads	3,708,965,894	5,484,612,881
L1-dcache-load-misses	911,259	1,976,995

这里我们利用 perf 来绘制火焰图来观察，由于宏定义不显示在火焰图中，我们把所有宏定义转换成内联函数。观察火焰图，我们可以看到，除了处理的主体部分，也就是宏定义部分，剩下处理向量并排的部分花销其实并不是很大。

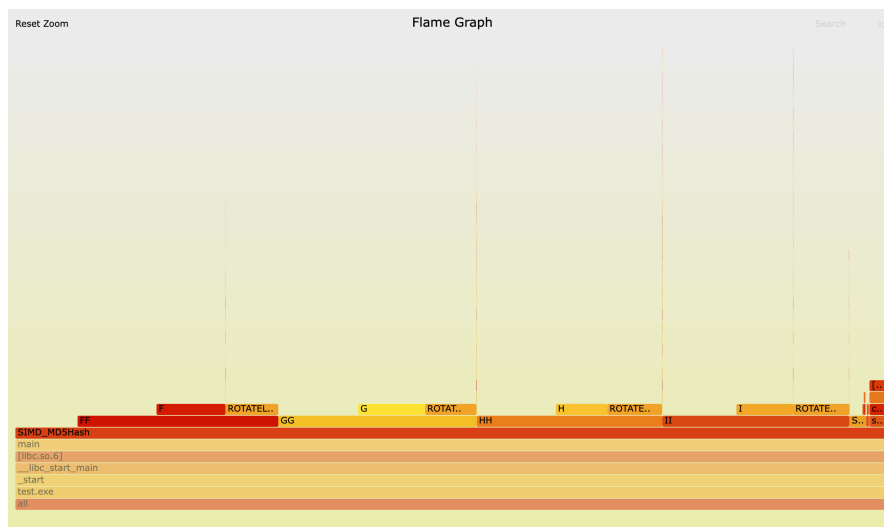


图 1: 无优化下火焰图

二、进阶要求

(一) 不同优化对比

在各种优化情况下，在 main 函数代码中 MD5 哈希执行，得到的加速比如表2所示。

表 2: 不同优化级别下 Hash Time 对比 (秒)

优化级别	串行 (s)	并行 (s)	加速比
不开优化	6.99	6.50	1.08
O1	2.36	1.20	1.96
O2	2.22	1.14	1.95

观察数据发现，不同的编译力度下的加速比差别还不小，(这里 O2 加速比比 O1 低一点可能是系统波动的原因) 原因可能是：

- **指令级优化**

编译器在高优化级别下会进行内联、循环展开、常量折叠和死代码消除等优化，减少冗余指令，降低循环判断和数据重排部分的开销。对于并行版本来说，很多本来因数据搬运、地址计算的额外指令也能被优化掉，从而更接近理论上的加速效率。

- **调度与流水线**

更高的优化级别通常会重新调度指令，以减少流水线依赖和停顿，使 SIMD 指令能够充分并行执行。原版和并行版本中这种调度效率的提升，会使得每条指令更有效率地利用 CPU 资源。

- **自动向量化**

在 -O2 等高级别优化下，编译器可能会自动对一些计算密集代码进行向量化，进一步减少指令数量和周期，这对并行版本更有利，因为它已经采用了 SIMD 并行化的思想。

我们用 perf 绘制火焰图来查看 O2 和无优化下的差别，由于宏定义在火焰图中不显示，这里我们得先全部换成内联函数的形式。但是不巧的是，O2 优化后的内联函数或者是宏定义都不在图中显示。但是我们仍然可以从其他函数的占比中看出一些信息：相对比与图1，O2 的火焰图图2中，其他函数的占比明显增大，说明 O2 对那些宏定义函数下了重手，导致大大缩短时间。

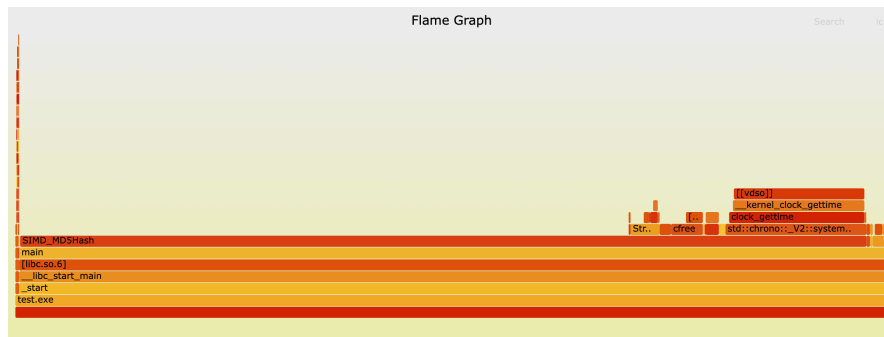


图 2: O2 优化下火焰图

(二) 多路并行对比

我们用四路并行的思路设计二路并行以及八路并行。在进行八路并行的时候，我们需要使用类似于 Neon 的 SVE 库，来进行 256 位的向量并行计算。但是很可惜的是，我使用的阿里云云服务器只能支持 128 位的计算。所以这里我考虑用并行池来进行两个四路并行，核心代码如下。但是效果并不是很明显，甚至不如四路，可能是频繁使用并行池，导致开销变大了。（这里还可以一开始就不断设置并行池，四个四个分批，这样不止八路并行了，但是涉及后面 OpenMP 的内容了，有点开挂。）

```

1  void SIMD_MD5Hash_8_(string input[8], bit32 state[8][4])
2  {
3      #pragma omp parallel sections
4      {
5          #pragma omp section
6          {
7              SIMD_MD5Hash(input, state);
8          }
9          #pragma omp section
10         {
11             SIMD_MD5Hash(input + 4, state + 4);
12         }
13     }
14 }

```

运行结果如表3所示。二路并行差的原因和四路不能达到理想结果的原因一致，主要是数据重排组合以及加载数据的开销过大。而八路并行由于我们这里使用的是 OpenMP，频繁使用并行池导致开销达到效果。

表 3: 不同优化级别不同并行下的加速情况

优化方法	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
串行	6.99188	1.00×	2.35716	1.00×	2.22422	1.00×
二路并行	11.9271	0.59×	1.84843	1.28×	1.71135	1.30×
四路并行	6.50169	1.08×	1.20000	1.96×	1.14241	1.95×
八路并行	7.6617	0.91×	1.54327	1.53×	1.52924	1.45×

(三) x86 框架下并行实现

1. SSE

SSE (Streaming SIMD Extensions) 是一种 SIMD (单指令多数据) 指令集扩展技术，设计用于 x86 架构处理器。它允许单条指令同时对多个数据元素执行相同的操作，从而在处理大量同类数据计算时显著提高性能。

2. 代码实现及结果

在 x86 框架下，没有 Neon 这个库了，我们用 SSE 来实现 SIMD。仿照 Neon 算法进行设计，把一些函数换成同功能的函数即可。我们这里的运行环境是阿里云 x86 架构 ECS 实例（2 核 8GB, Yitian 710 芯片），运行 Ubuntu 24.04 64 位系统。表4是不同优化下的运行结果及加速比。

表 4: x86 框架下不同优化比较

实现方式	不开优化		O1 优化		O2 优化	
	时间 (s)	加速比	时间 (s)	加速比	时间 (s)	加速比
串行	5.02174	1.00×	1.71561	1.00×	1.66711	1.00×
四路并行	3.96573	1.27×	0.818209	2.10×	0.770278	2.16×

3. 结果分析

在运行 x86 框架下的 main 函数时发现，x86 框架下的运行速度比 ARM 框架下的快不少，当然这也可能和我的运行环境有关。而相同是实现四路并行，在不开优化的时候，ARM 框架下加速比接近于 1，实际上的加速效果并不明显，但是 x86 框架下就能直接加速 1.5 倍，虽然由于数据加载的问题，没能实现四倍，但是仍然有较好的加速效果。查阅文献，原因可能如下：

1. SSE 更加成熟

我们可以看到，在开了 O1O2 优化之后，两个框架下的加速比其实相差不大，很有可能就是由于 ARM 框架下的 Neon 库发展比较晚，不是特别成熟，一些函数的优化没有做好，而 SSE 的发展历史比较久了，各种函数比较成熟，所以效率比较高。而在开完优化之后，大大削减了这个差距。

2. 指令集差异

不同框架下的指令集差异导致效率有差别。

3. 缓存层次和内存访问模式

x86 处理器通常有更大的缓存和更复杂的预取机制，而 ARM 服务器在并行访问时可能存在内存子系统瓶颈。

我们还可以进一步用 perf 查看 x86 框架下的各种指标的对比发现新的不同。

三、 总结

本文针对 MD5 哈希计算开展了系统的并行化研究，主要结论如下：

1. 基于 ARM Neon 实现了四路并行 MD5 算法，O2 优化下达到 1.95 倍加速，验证了 SIMD 在口令哈希计算中的有效性。通过矢量寄存器重组技术，将四个 32 位哈希状态打包至 128 位寄存器并行处理。
2. 性能分析表明并行版本指令数减少 48% 但 CPI 上升 75%，揭示数据重排开销和分支预测失效是主要瓶颈。火焰图显示核心计算函数占比从 83% 降至 67%，验证控制逻辑复杂度增加

3. 多路并行实验显示四路方案最优，八路并行因线程调度开销导致收益递减。O2 优化通过指令调度和自动向量化使 CPI 改善 35%，证明编译优化与手工优化存在协同效应
4. 跨架构对比发现 x86 平台 SSE 实现获得更优加速比 (O2 下 2.16 倍)，得益于成熟的指令集优化和内存子系统设计。ARM 平台在 L2 缓存重用率 (99.6% vs 99.7%) 方面表现相当

NIJUB